OPERATION KOMOREBI

Komorebi Circuit Obstructions

Lean-Verified Core

Block 01: restored computation
and complete cut-discrepancy covers

Komorebi P.

Local source release v1.1.0 • September 21, 2026

Formal coverage, not a change to the research scope

Forty checked declarations support maintained claim C01 and Theorem 2.1 of the original 28-page companion. The other seven maintained claim groups and the other 28 numbered entries are not formalized by this block.

Exact source statements, source locations, checked examples, build evidence, and proof boundaries. No resolution of P versus NP or independent mathematical review is claimed.

What is now formally proved

Let $C = Q \circ P$ be a finite legal classical NOT/CNOT/Toffoli circuit satisfying, for every input x and both initial target bits b,

$$C(x, 0^a, b) = (x, 0^a, b \oplus f(x)).$$

At any cut, define the complete register discrepancy

$$H_i(x) = P(x, 0^a, 0)_i \oplus Q^{-1}(x, 0^a, 0)_i.$$

The formal development proves

Maintained result C01

$$H_i(x) = 1 \Longrightarrow f(x) = 1, \qquad f(x) = 1 \Longleftrightarrow \exists i\ H_i(x) = 1.$$

Equivalently, every packet is safe and the packets together cover every positive input. For the finite register set, $f(x) = \bigvee_i H_i(x)$.

The quantifiers cover every finite input size, every finite zero-workspace size, every legal circuit satisfying the endpoint condition, every logical input, and every cut index. Intermediate workspace may be dirty; input registers may change temporarily; the designated output may be used as a control.

Exact formal endpoint

`CircuitObstructions.canonical_complete_safe_cover`

The generic theorem `CircuitObstructions.complete_safe_cover` works for any Boolean register encoding with a designated target that truly stores b. The canonical specialization instantiates precisely the finite $(x, 0^a, b)$ model.

Maintained claims	1 of 8 formalized: C01.
Original companion	Theorem 2.1 of 29 numbered entries.
Formal inventory	16 gate/circuit lemmas, 13 boundary/cut lemmas, 11 example and control theorems.
Compiler	Lean 4.34.0-rc2, bundled standard library only.
Audit	Fresh build, strict warnings, axiom inventory, and bundled checker replay.

Forty declarations are not forty independent discoveries. Many are support lemmas or explicit model controls. The accompanying Lean sources contain every proof term; the statement register below covers all 40 declarations without including imported library theorems.

The model and the proof

DEFINITIONS USED IN EVERY REGISTER ENTRY

A state is $s : W \to \{0, 1\}$, where W has decidable equality. Write $s[t \leftarrow b]$ for replacement of coordinate t by b. The primitive gates are

$$\begin{aligned} N_t(s) &= s[t \leftarrow \neg s_t], \\ C_{c,t}(s) &= s[t \leftarrow s_t \oplus s_c], \qquad c \neq t, \\ T_{a,b,t}(s) &= s[t \leftarrow s_t \oplus (s_a \wedge s_b)], \end{aligned}$$

with three pairwise distinct wires in $T_{a,b,t}$. These inequalities are fields of the Lean gate constructors, not comments checked only by a parser.

A circuit is a list of gates, executed from left to right. We write $p; q$ for list append and $\mathrm{rev}(p)$ for list reversal. Because each gate is self-inverse, reversal is a proved circuit inverse.

An encoding $\mathrm{Enc}(x, b)$ has a named target t with $\mathrm{Enc}(x, b)_t = b$. The predicate $\mathrm{Restores}(\mathrm{Enc}, c, f)$ is exactly

$$\forall x, b, \quad \mathrm{run}(c, \mathrm{Enc}(x, b)) = \mathrm{Enc}(x, b \oplus f(x)).$$

It is a hypothesis defining which circuits implement the specified oracle. It does not assume the discrepancy-cover conclusion.

Derivation at an arbitrary cut

Apply restoration to initial target $b \oplus f(x)$ and undo the suffix. Circuit reversal and XOR cancellation give

$$\mathrm{run}(\mathrm{rev}(q), \mathrm{Enc}(x, b)) = \mathrm{run}(p, \mathrm{Enc}(x, b \oplus f(x))).$$

Put $A_i = \mathrm{run}(p, \mathrm{Enc}(x, 0))_i$ and

$$D_i = \mathrm{run}(p, \mathrm{Enc}(x, 1))_i \oplus A_i.$$

The states $\mathrm{Enc}(x, 0)$ and $\mathrm{Enc}(x, 1)$ differ at their target. Injectivity of p therefore gives $D_i = 1$ for at least one coordinate. Substituting $b = 0$ into the inverse-suffix identity gives

$$H_i = f(x) \wedge D_i.$$

If $f(x) = 0$, every discrepancy vanishes. If $f(x) = 1$, a nonzero direction coordinate supplies a discrepancy. This proves simultaneous safety and completeness.

No hidden cost theorem

This block formalizes the semantic identity, reversal laws, and example counts. It does not formalize polynomial degree, minimal gate complexity, safe-packet degree, the ordinary-circuit simulation bounds, or any superpolynomial lower bound.

State updates and Boolean identities

GATES.LEAN / DECLARATIONS G01–G06

G01 Read the updated wire

`CircuitObstructions.write_same` [line 17]

$s[t \leftarrow b]_t = b$ for every state, target, and Boolean value.

G02 Other wires are unchanged

`CircuitObstructions.write_other` [line 20]

If $i \neq t$, then $s[t \leftarrow b]_i = s_i$.

G03 Right XOR cancellation

`CircuitObstructions.xor_cancel_right` [line 23]

$(a \oplus b) \oplus b = a$ for all $a, b \in \{0, 1\}$.

G04 Left XOR cancellation

`CircuitObstructions.xor_cancel_left` [line 26]

$a \oplus (a \oplus b) = b$ for all $a, b \in \{0, 1\}$.

G05 XOR commutativity

`CircuitObstructions.xor_commute` [line 29]

$a \oplus b = b \oplus a$ for all Boolean a, b.

G06 XOR detects inequality

`CircuitObstructions.xor_true_iff_ne` [line 32]

$a \oplus b = 1$ if and only if $a \neq b$.

Why these elementary lemmas are exposed

The proof audit prints elaborated statement types, not just theorem names. A development attempt used overloaded infix notation that did not elaborate to the intended equality. The checked version uses explicit `Bool.xor` in the formal source. The mathematical formulas above are the intended and checked statements.

The proofs of G03–G06 reduce the Boolean cases inside Lean. They do not use external truth-table output as a proof certificate. Functional update lemmas are proved from the definition of `write`.

Reversal is proved from legal gates

GATES.LEAN / DECLARATIONS G07–G14

G07 Each legal gate is self-inverse

`CircuitObstructions.Gate.eval_involutive` [line 48]

$g(g(s)) = s$. Distinct control and target wires are part of g.

G08 Each gate is injective

`CircuitObstructions.Gate.eval_injective` [line 67]

$g(s) = g(t) \Longrightarrow s = t$.

G09 Empty execution

`CircuitObstructions.run_nil` [line 78]

$\mathrm{run}([], s) = s$.

G10 Execution of a leading gate

`CircuitObstructions.run_cons` [line 81]

$\mathrm{run}(g :: p, s) = \mathrm{run}(p, g(s))$.

G11 Composition convention

`CircuitObstructions.run_append` [line 84]

$\mathrm{run}(p; q, s) = \mathrm{run}(q, \mathrm{run}(p, s))$.

G12 Reverse after forward

`CircuitObstructions.run_reverse_run` [line 90]

$\mathrm{run}(\mathrm{rev}(c), \mathrm{run}(c, s)) = s$.

G13 Forward after reverse

`CircuitObstructions.run_run_reverse` [line 98]

$\mathrm{run}(c, \mathrm{run}(\mathrm{rev}(c), s)) = s$.

G14 Every legal circuit is injective

`CircuitObstructions.run_injective` [line 102]

$\mathrm{run}(c, s) = \mathrm{run}(c, t) \Longrightarrow s = t$.

G07 uses the update rules and cancellation. G12 is induction on the gate list using G07 and G11; G13 and G14 follow from the inverse identity. No external permutation oracle is assumed.

Counts and boundary coordinates

GATES.LEAN / CUTCOMPARISON.LEAN

G15 Toffoli counts add

`CircuitObstructions.toffoliCount_append` [line 114]

$\mathrm{tc}(p; q) = \mathrm{tc}(p) + \mathrm{tc}(q)$. A Toffoli has cost one; NOT and CNOT have cost zero for this statistic.

G16 Reversal preserves the count

`CircuitObstructions.toffoliCount_reverse` [line 118]

$\mathrm{tc}(\mathrm{rev}(c)) = \mathrm{tc}(c)$. This is an exact list-count identity, not a lower bound.

B01 The two target branches are distinct

`CircuitObstructions.Boundary.target_separated` [line 17]

$\mathrm{Enc}(x, 0) \neq \mathrm{Enc}(x, 1)$, because their designated target values differ.

B02 Canonical logical input

`CircuitObstructions.canonical_input` [line 39]

On the input wire indexed by $i \in \mathrm{Fin}(n)$, $\mathrm{Enc}(x, b)_i = x_i$.

B03 Canonical workspace initialization

`CircuitObstructions.canonical_ancilla` [line 42]

On every workspace wire indexed by $j \in \mathrm{Fin}(a)$, $\mathrm{Enc}(x, b)_j = 0$.

B04 Canonical target initialization

`CircuitObstructions.canonical_target` [line 45]

The unique designated target has value b.

The canonical wire type is

$$\mathrm{Fin}(n) \sqcup (\mathrm{Fin}(a) \sqcup \mathbf{1}).$$

This covers zero inputs or zero ancillary wires as well as arbitrary positive sizes. It imposes zero workspace only at the endpoints, not during the gate list.

The exact transport identity

CUTCOMPARISON.LEAN / DECLARATIONS B05–B07

Throughout B05–B13, $p; q$ is legal and Restores(Enc, $p; q, f$) holds. The symbols A_i, D_i, H_i have the definitions on page 3.

B05 The inverse suffix meets the correct target branch

`CircuitObstructions.inverse_suffix_boundary` [line 49]

For every x, b,

$$\mathrm{run}(\mathrm{rev}(q), \mathrm{Enc}(x, b)) = \mathrm{run}(p, \mathrm{Enc}(x, b \oplus f(x))).$$

B06 The transported target direction cannot disappear

`CircuitObstructions.direction_nonzero` [line 67]

For every x, there exists a wire i with $D_i(x) = 1$. This theorem needs only p and the separated encoding, not restoration of a whole circuit.

B07 Discrepancy equals output times direction

`CircuitObstructions.cut_transport_identity` [line 82]

For every x, i,

$$H_i(x) = f(x) \wedge D_i(x).$$

How the assumptions are used

B05 uses the restoration equation and the proved inverse of q. B06 uses only target separation and injectivity of p. B07 splits on the Boolean value $f(x)$ and substitutes B05.

This separates an actual circuit fact from its specification. A different function f can be substituted only after proving the corresponding endpoint equation. The theorem does not claim that a restored implementation of an arbitrary f is small or easy to find.

The complete state matters

The coordinate witnessing $D_i = 1$ need not be the originally designated target. The output-active example on page 11 places every useful discrepancy in a data wire at the chosen cut.

Safety and complete coverage

CUTCOMPARISON.LEAN / DECLARATIONS B08–B11

B08 Every discrepancy is safe

`CircuitObstructions.cut_packet_safe` [line 91]

For all x, i, $H_i(x) = 1 \Longrightarrow f(x) = 1$.

B09 Every positive input is detected

`CircuitObstructions.cut_cover_complete` [line 99]

For each x,

$$f(x) = 1 \Longleftrightarrow \exists i\ H_i(x) = 1.$$

B10 Safety and completeness in one declaration

`CircuitObstructions.complete_safe_cover` [line 111]

The conjunction of

$$\big(\forall x, i,\ H_i(x) = 1 \Rightarrow f(x) = 1\big)$$

and

$$\big(\forall x,\ f(x) = 1 \Longleftrightarrow \exists i\ H_i(x) = 1\big).$$

B11 Equality characterizes rejection

`CircuitObstructions.cut_states_equal_iff_reject` [line 119]

For every x,

$$\mathrm{run}(p, \mathrm{Enc}(x, 0)) = \mathrm{run}(\mathrm{rev}(q), \mathrm{Enc}(x, 0))$$
$$\Longleftrightarrow \quad f(x) = 0.$$

B08 follows directly from B07. For B09, an accepted input and the witness supplied by B06 give a true discrepancy through B07. The reverse implication is B08. B10 packages these universal statements. B11 follows by the inverse-suffix identity and target separation.

Scope alignment. B06, B07, and B10 collectively discharge the mathematical statement of original companion Theorem 2.1. The existence form of the cover is equivalent to its finite OR formulation. No numerical degree argument is part of this alignment.

Every cut, in the canonical finite model

CUTCOMPARISON.LEAN / DECLARATIONS B12–B13

B12 Cut positions are arbitrary

`CircuitObstructions.every_index_cut` [line 131]

For every $k \in \mathbb{N}$ and x, take $p = c.\mathrm{take}(k)$ and $q = c.\mathrm{drop}(k)$. Then $f(x) = 1 \iff \exists i\ H_i(x) = 1$.

B13 Complete safe coverage for finite zero-workspace oracles

`CircuitObstructions.canonical_complete_safe_cover` [line 139]

For all n, a, c, f satisfying canonical restoration and for every cut index k, the simultaneous safety and completeness statement B10 holds for all $x : \mathrm{Fin}(n) \to \{0, 1\}$ and all canonical wires.

Indices past the end of the list are allowed: `take` gives the whole circuit and `drop` the empty circuit. The identity `take k ++ drop k = c` supplies restoration at every index.

Exact source type of the principal specialization

```
theorem canonical_complete_safe_cover (n a :  Nat)
  (c :  Circuit (CanonicalWire n a))
  (f :  (Fin n → Bool) → Bool)
  (hR : Restores (canonicalBoundary n a) c f) (k :  Nat) :
  (∀ x i,
      packet (canonicalBoundary n a) (c.take k) (c.drop k) x i
      = true → f x = true) ∧
  (∀ x, f x = true ↔
      ∃ i, packet (canonicalBoundary n a)
          (c.take k) (c.drop k) x i = true)
```

The line wrapping above is editorial. The authoritative type and proof are in the named source, and the elaborated types are retained in the audit log.

The verified bridge is semantic

This theorem identifies a complete safe discrepancy cover produced by any correctly restored legal circuit. It does not prove that the cover has high degree or that constructing it is expensive. C02 and its degree prerequisites are explicitly outside Block 01.

A checked dirty-workspace example

EXAMPLES.LEAN / DECLARATIONS E01–E04

Use four wires (u, v, s, y), with $s = 0$ initially. The circuit is

$$T_{u,v,s} \ ; \ C_{s,y} \ ; \ T_{u,v,s}.$$

It first stores $u \wedge v$ in the scratch wire, copies that value to the target by XOR, and uncomputes the scratch wire. The input bits never need to be restricted to a chosen example for the restoration theorem.

E01 The AND oracle restores its registers

`CircuitObstructions.Examples.andOracle_restores` [line 27]

For all u, v, y,

$$(u, v, 0, y) \longmapsto (u, v, 0, y \oplus (u \wedge v)).$$

E02 The displayed implementation has two Toffolis

`CircuitObstructions.Examples.andOracle_toffoli_count` [line 33]

$\mathrm{tc}(\texttt{andOracle}) = 2$. This count is not an optimality claim: a direct Toffoli into the target also computes AND.

E03 Workspace is genuinely nonzero at an intermediate step

`CircuitObstructions.Examples.and_scratch_can_be_dirty` [line 36]

On $(u, v, y) = (1, 1, 0)$, the scratch wire is 1 after the first gate.

E04 The complete cover holds at every cut

`CircuitObstructions.Examples.andOracle_every_cut` [line 40]

For all cut indices k and both-bit inputs (u, v),

$$u \wedge v = 1 \iff \exists i \ H_i(u, v) = 1.$$

These example proofs reduce every Boolean case in Lean and instantiate the universal theorem. The proof of C01 itself is not a finite enumeration over circuit sizes.

The discrepancy can leave the target

EXAMPLES.LEAN / DECLARATIONS E05–E10

On data wire d and target y, implement SWAP with three CNOTs. Let

$$C = \mathrm{SWAP}\,;\ N_d\ ;\ \mathrm{SWAP}\,.$$

This seven-gate circuit temporarily uses the target as a control and returns (x, y) as $(x, \neg y)$. Cut immediately after the first SWAP.

E05 Restoration despite target feedback

`CircuitObstructions.Examples.outputActiveOracle_restores` [line 66]

Restores(`swapBoundary`, C, $x \mapsto 1$).

E06 Seven gates in this explicit construction

`CircuitObstructions.Examples.outputActiveOracle_length` [line 71]

$|C| = 7$. It is an example circuit, not a minimum-size realization.

E07 No nonlinear gates are needed here

`CircuitObstructions.Examples.outputActiveOracle_toffoli_count` [line 73]

$\mathrm{tc}(C) = 0$.

E08 The data discrepancy is always true

`CircuitObstructions.Examples.moved_discrepancy_data` [line 76]

At the specified cut, $H_d(x) = 1$ for every $x \in \{0, 1\}$.

E09 The target discrepancy is always false

`CircuitObstructions.Examples.moved_discrepancy_target` [line 80]

At that same cut, $H_y(x) = 0$ for every $x \in \{0, 1\}$.

E10 Target-only reconstruction is refuted

`CircuitObstructions.Examples.target_only_reconstruction_is_false` [line 85]

$\neg(\forall x \in \{0, 1\},\ H_y(x) = 1)$. Since f is identically one, this is a concrete failure of replacing the complete register comparison by the named target alone.

All six statements are checked propositions, not external simulation reports. They test the exact distinction between output-active computation and a protected-output simplification.

A necessary boundary condition

EXAMPLES.LEAN / DECLARATION E11

E11 Ignoring the target makes restoration vacuous

`CircuitObstructions.Examples.degenerate_encoding_control` [line 93]

Let $\mathrm{bad}(b)$ be the all-zero state, independent of b. Then

$$\forall b, \quad \mathrm{run}([], \mathrm{bad}(b)) = \mathrm{bad}(b \oplus 1),$$

while

$$\forall i, \quad \mathrm{bad}(0)_i \oplus \mathrm{bad}(0)_i = 0.$$

The constant encoding satisfies a restoration-shaped equation for the wrong constant function without producing any discrepancy. It is deliberately not a valid `Boundary`: no designated target can read back both supplied Boolean values.

This control explains why `Boundary.target_eq` and B01 are substantive hypotheses, not bureaucratic encoding details.

Four deliberate rejection tests

In addition to the positive library, the runner creates four invalid test files in its isolated build directory. None is imported into the theorem library.

False proposition	An attempted proof of `False` must not compile.
Illegal gate	A CNOT with identical control and target must not satisfy its constructor proof.
Wrong reconstruction	The false target-only statement from the SWAP example must be rejected.
Admitted proof	An intentional `sorry` must be rejected by the warning-as-error guard.

All four were rejected in the recorded verification. Rejection of these controls is a check on the pipeline; it does not replace the proof of any universal theorem.

What these tests do not certify

They are not an exhaustive audit of Lean or its kernel. The compiler and bundled checker come from the same pinned toolchain. No separate kernel implementation or independent human mathematical review was used.

Verification, provenance, and reproduction

EXACT EVIDENCE BOUNDARIES

Pinned implementation

Lean 4.34.0-rc2, `x86_64-unknown-linux-gnu`, release commit `6a10ac8c22beadecabdbb0919c2b50214762f91d`. Only its bundled standard library is imported; there is no Mathlib dependency. A fresh source copy is built with `lake -wfail build`. Each module is also checked with warnings promoted to errors. The bundled `leanchecker` replays all three modules.

Axiom and statement audit

All 40 source declarations are matched to elaborated statements and printed axiom dependencies. The union of dependencies is

$$\{\texttt{propext}, \texttt{Classical.choice}, \texttt{Quot.sound}\}.$$

These are standard Lean axioms. No active admitted proof, custom axiom, `native_decide`, or kernel-check bypass is accepted by the runner. Standard axioms are not described as an axiom-free proof. Logs and the exact source inventory are delivered with the repository.

Preserved scientific evidence

The original full active Python/C++ suite passed, including 17 unit-test methods and all seven frozen scientific-output comparisons. Those finite results remain separate from the Lean proof. A new full recursive replay of all ten historical research archives is not claimed for this block.

Reproduce after installing the pinned toolchain

From the repository root:

```
python scripts/verify_lean.py --toolchain /path/to/toolchain
python scripts/verify.py --full
```

The formal runner creates a fresh source build, audits axioms, runs checker replay and the four negative controls, and records hashes. The second command needs a C++17 compiler in addition to Python. Neither command installs dependencies or contacts the network.

Recovered compiler and retained failures

Direct compiler download failed because outbound access was blocked. The usable toolchain was recovered from the user's `Lean_Compact_01.zip`. This was a partial archive extraction: the complete multipiece portable package was not reassembled or authenticated. The required compiler, standard library, and checker ran successfully; their byte hashes are recorded.

A Boolean-notation ambiguity and a comment-sensitive declaration parser were caught and corrected before the final formal inventory. Failed attempts are retained. No failed build is labelled verified. This local release does not publish, archive, or alter a hosted GitHub repository.

Complete declaration index

SOURCE LOCATIONS IN THE PINNED LEAN BLOCK

Each row names exactly one active theorem. Names below omit the common prefix `CircuitObstructions.`; example names retain their `Examples.` namespace. G, B, and E identify the statement entries in this reference. File lines are physical source lines.

ID	Lean declaration	Line
G01	`write_same`	17
G02	`write_other`	20
G03	`xor_cancel_right`	23
G04	`xor_cancel_left`	26
G05	`xor_commute`	29
G06	`xor_true_iff_ne`	32
G07	`Gate.eval_involutive`	48
G08	`Gate.eval_injective`	67
G09	`run_nil`	78
G10	`run_cons`	81
G11	`run_append`	84
G12	`run_reverse_run`	90
G13	`run_run_reverse`	98
G14	`run_injective`	102
G15	`toffoliCount_append`	114
G16	`toffoliCount_reverse`	118
B01	`Boundary.target_separated`	17
B02	`canonical_input`	39
B03	`canonical_ancilla`	42
B04	`canonical_target`	45
B05	`inverse_suffix_boundary`	49
B06	`direction_nonzero`	67
B07	`cut_transport_identity`	82
B08	`cut_packet_safe`	91
B09	`cut_cover_complete`	99
B10	`complete_safe_cover`	111
B11	`cut_states_equal_iff_reject`	119
B12	`every_index_cut`	131
B13	`canonical_complete_safe_cover`	139

G entries are in `CircuitObstructions/Gates.lean`; B entries are in `CircuitObstructions/CutComparison.lean`. Every source file is relative to `formalization/lean/`.

Examples and source identity

COMPLETE DECLARATION INDEX, CONTINUED

The final eleven declarations are in `CircuitObstructions/Examples.lean`. They instantiate the universal result, check dirty and output-active behavior, or refute explicitly stated invalid shortcuts.

ID	Lean declaration	Line
E01	`Examples.andOracle_restores`	27
E02	`Examples.andOracle_toffoli_count`	33
E03	`Examples.and_scratch_can_be_dirty`	36
E04	`Examples.andOracle_every_cut`	40
E05	`Examples.outputActiveOracle_restores`	66
E06	`Examples.outputActiveOracle_length`	71
E07	`Examples.outputActiveOracle_toffoli_count`	73
E08	`Examples.moved_discrepancy_data`	76
E09	`Examples.moved_discrepancy_target`	80
E10	`Examples.target_only_reconstruction_is_false`	85
E11	`Examples.degenerate_encoding_control`	93

The source modules are `CircuitObstructions/Gates.lean` (G), `CircuitObstructions/CutComparison.lean` (B), and `CircuitObstructions/Examples.lean` (E), relative to `formalization/lean/`.

Source identity and maintained scope

The unchanged predecessor is tagged v1.0.0 at commit `caa9e3be9af8e0dc8ba8f44726f9d28e81f93f20`. The present additive formal block is pinned by `theorem_inventory.json`, source hashes in `evidence/lean-block 01/verified/verification.json`, and the release's integrity seal. The original companion and source archive are preserved unchanged.

Remaining formalization boundary. C02–C08, ordinary-circuit simulation, multiplicative degree, algebraic lower bounds, polynomial projection, and faithful-algebra dimension arguments remain written/computational material. Their appearance in the original book does not acquire Lean status by association with C01.

References

The mathematical source is `docs/FOUNDATIONS.md`, claim C01 and original companion Theorem 2.1. The exact executable formal source is the three-module inventory above. For Lean's axiom conventions, see the official manual, *Theorem Proving in Lean 4*, "Axioms and Computation," https://lean-lang.org/theorem_proving_in_lean4/Axioms-and-Computation/. No novelty or independent-review claim is made.